Series DB

Superblocks (B0, B1)

Schema table (S)

Record blocks (R)

Continuation blocks (C)

Series DB

Flash storage is divided into blocks. Each block can store 4094 bytes of data.

When a database is accessed, it needs at least two things:

- **Magic number:** Identifies the start of a header and indicates that the information for that header begins at this address.
- **CRC checksum:** Detects accidental changes or corruption in raw data. A mathematical operation reduces each set of data—for example, 250 bytes to a smaller checksum. Compare the checksums for 2 sets to check whether the data is the same, `without comparing all 250 bytes one by one`:

  ```
  Set 1: 250 bytes of data → 2-byte checksum 1
  Set 2: 250 bytes of data → 2-byte checksum 2
  ```

  If the checksums match, the data is likely the same; if they differ, the data is different or corrupted.

Superblocks (B0, B1)

The 2 fixed blocks used to configure the database are copies of each other. If one block is corrupted, for example, its `magic number` or `CRC` does not match, the other block can be used to recover the configuration.

The superblock also needs a `version` number. It identifies the expected format and allows the format to change over time, such as from version 1 to version 2.

Each superblock contains:

field	size	purpose
magic_number* (`M`)	2 bytes	Identifies the start of the superblock header.
crc_16(*) (`C`)	2 bytes	Detects accidental changes or corruption in the superblock data.
version* (`V`)	1 byte	Identifies the superblock format.
flags (`F`)	1 byte	Stores superblock flags.
schema_table (`S`)	Variable	Maps schema IDs to payload layouts.

> In the field lists, `*` means the field is required. `(*)` means the field is required, but its size still needs to be determined.

This shows how the superblock is stored in 2 copies, B0 and B1. Each copy contains the same fields:

```
[B0: M, V, C, F, S]
[B1: M, V, C, F, S]
```

Schema table (S)

The schema table lives in the `superblock`. It answers this question when reading: "Given a `schema_id`, what is the byte layout of the payload?"

For example, say the payload is 4 bytes and the schema says the first 2 bytes are temperature and the next 2 are humidity:

```
 payload     = 0x01 0x02 0x03 0x04
temperature = 0x01 0x02
humidity    = 0x03 0x04
```

The schema table contains a table header followed by schema entries:

```
 B0 (M, V, C, F)
└── Schema table (S)
    ├── Table header (Th)
    ├── Entry 0 (T_e0)
    ├── Entry 1 (T_e1)
    ├── Entry 2 (T_e2)
    └── ...
```

The table header (Th) contains:

field	size	purpose
crc_16(*)	2 bytes	Checksum calculated over all schema entries.
schema_count*	1 byte	Number of schema entries in the table.
table_length*	1 byte	Length of the schema table.

A schema entry (S_en) contains:

field	size	purpose
schema_id*	1 byte	Identifies the schema entry.
column_count*	1 byte	Number of columns in the schema.
column_widths(*)	8 bytes	Stores the width of each column.
column_signs(*)	8 bits	One bit per column, from most to least significant; `1` = signed, `0` = unsigned.

For example:

```
 schema_id     = 0
column_count  = 3
column_widths = [8, 16, 32]
column_signs  = 0b01000000
```

This entry describes 3 columns:

- Column 1: unsigned 8-bit value.
- Column 2: signed 16-bit value that can hold a negative number.
- Column 3: unsigned 32-bit value.

In summary, one database has one schema table. The table has one table header and can contain many schema entries.

```
 B0 (M, V, C, F)
└── Schema table (S)
    ├── Table header (Th)
    |   ├── CRC-16 (2 bytes)
```

```
|   ├── Schema count (1 byte)
|   └── Table length (1 byte)
├── Entry 0 (T_e0)
|   ├── Schema ID (1 byte)
|   ├── Column count (1 byte)
|   ├── Column widths (8 bytes)
|   └── Column signs (8 bits)
├── Entry 1 (T_e1)
|   └── ...
└── ...
```

Each extent carries a `schema_id` . The decoder looks up the ID to determine where each column is located in the payload. When the schema changes, existing extents keep their original IDs and can still be decoded with their original layouts.

> New data uses the new schema ID and **must be written to a new block**. A schema change is a hard boundary: the old extent ends and a new one begins. Since an extent starts with a record block, starting one always requires writing at least one new block.

Record blocks (R)

Record blocks (`R0` – `Rn`) follow superblocks `B0` and `B1` . They store records, such as a series of temperature readings.

A record block contains:

field	size	purpose
magic_number*	2 bytes	Identifies the start of the record block.
crc_16(*)	2 bytes	Detects accidental changes or corruption in the block.
schema_id(*)	2 bytes	Identifies the schema used to interpret the payload.
flags(*)	8 bits	Stores record block flags.
timing_config(*)	8 bytes	Stores the timing settings used to interpret record timestamps.
payload	Variable	Stores the records.

Continuation blocks (C)

Continuation blocks store records that follow a record block `R` when its payload is full. When a continuation block `C` fills, another continuation `C_cont` block is added. The sequence ends when a new record block `R_new` is written, such as after a schema change or recovery from power loss. Continuation blocks avoid repeating the record-block metadata for every block of the same extent.

A continuation block contains:

Field	Size	Purpose
magic_number*	2 bytes	Identifies the start of the continuation block.
crc_16(*)	2 bytes	Detects accidental changes or corruption in the block.
payload	Variable	Stores records that continue from the preceding block.

Each record block is followed by any continuation blocks needed to store its remaining records.

```
 [B0] [B1]
[R0] [R0-C0] [R0-C1] [R0-C2] ...
```

```
[R1] [R1-C0] [R1-C1] [R1-C2] ...
[R2] [R2-C0] [R2-C1] [R2-C2] ...
...
```

Each bracket represents one 4 KiB block.